benchmark_results_executed

April 1, 2026

1 ScyllaDB Vector Search Benchmark — 10M × 768 Dim (Cohere)

Dataset: 10M vectors · **768 dimensions** · COSINE similarity (Cohere)
Workload: Search Performance Test (VectorDBBench)
Date: 2026-04-01

1.0.1 Hardware

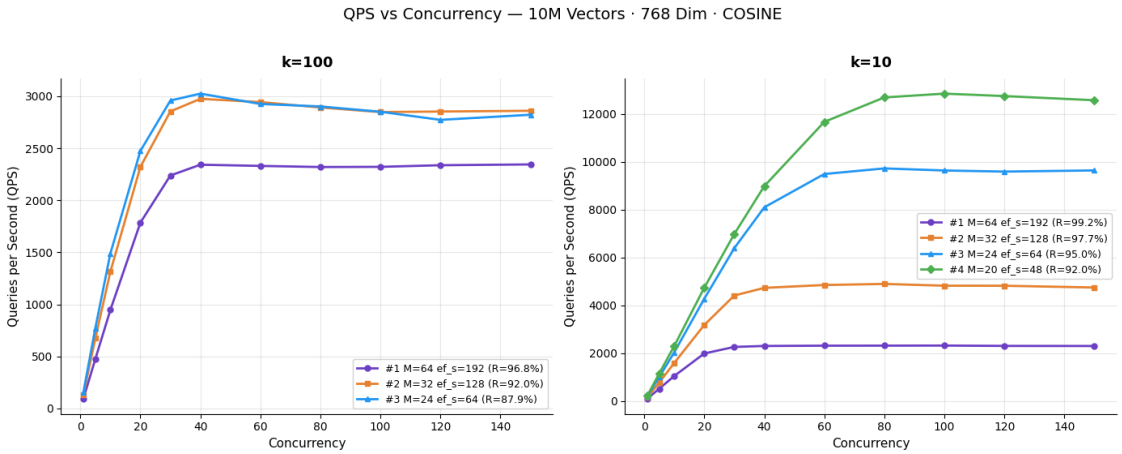
Role	Instance	Count
Storage nodes	i8g.large (16 GB RAM, 2 CPUs)	3
Search nodes	r7g.2xlarge (64 GB RAM, 8 CPUs)	2

1.0.2 Experiments

Experiment	M	ef_construction	ef_search	k tested
#1 (high quality)	64	384	192	100, 10
#2 (balanced)	32	256	128	100, 10
#3 (high throughput)	24	256	64	10, 100
#4 (max throughput)	20	256	48	10

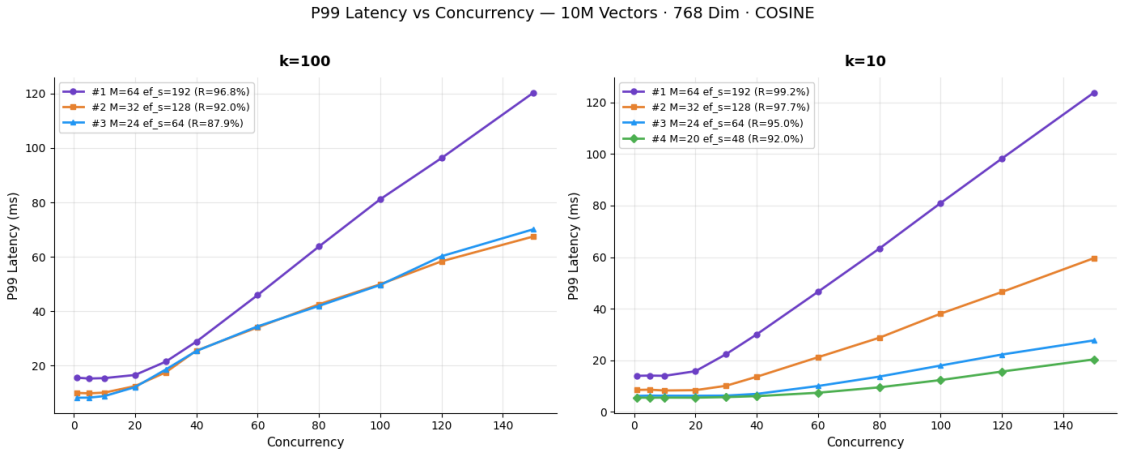
1.1 1. QPS vs Concurrency

How throughput scales with the number of concurrent search clients.

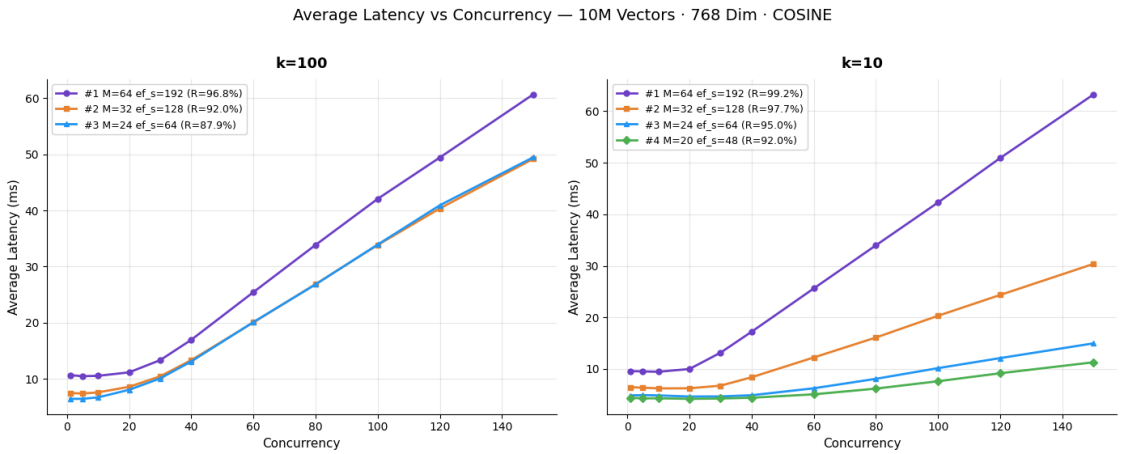


1.2 2. P99 Latency vs Concurrency

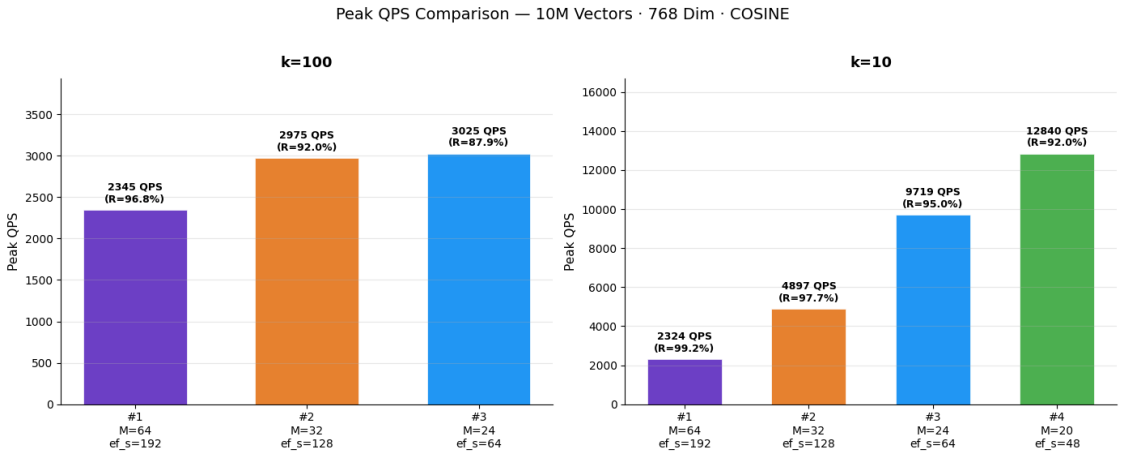
Tail latency behavior under increasing load.



1.3 3. Average Latency vs Concurrency

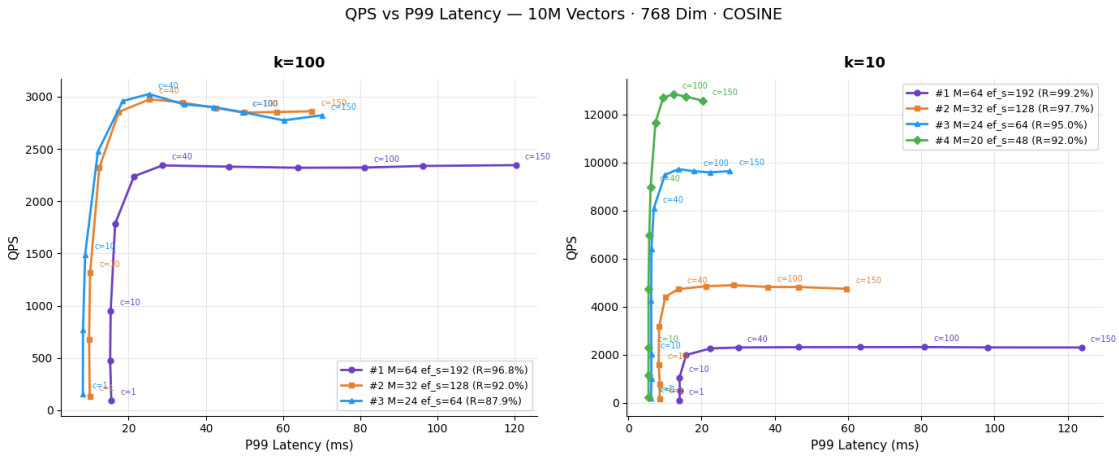


1.4 4. Peak QPS Comparison



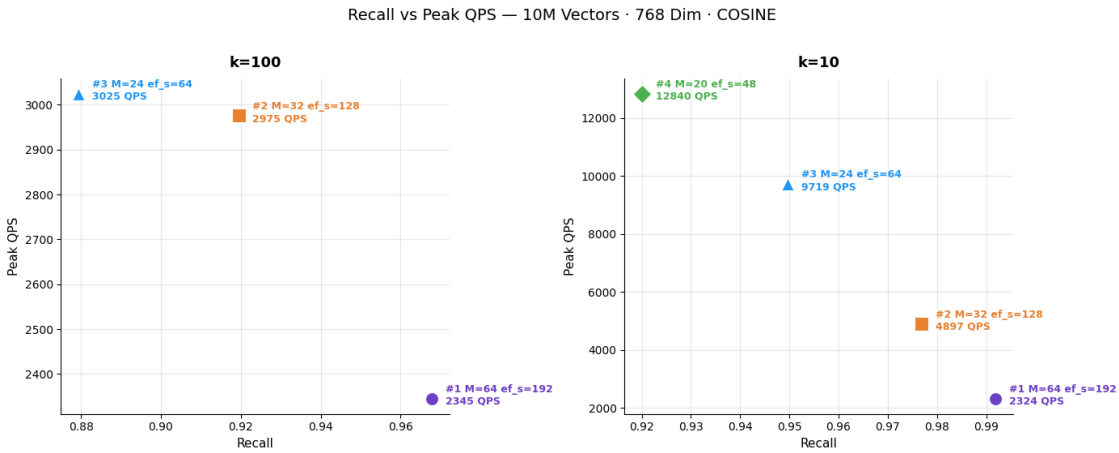
1.5 5. QPS vs P99 Latency (Pareto View)

Trade-off between throughput and tail latency. Points closer to the **top-left** corner are better (high QPS, low latency). Concurrency level annotated at key points.



1.6 6. Recall vs Peak QPS Trade-off

How index parameters affect the recall-throughput trade-off.



1.7 7. Summary

1.7.1 k=100

Metric	#1 M=64 ef_s=192	#2 M=32 ef_s=128	#3 M=24 ef_s=64
Peak QPS	2,345 (c=150)	2,975 (c=40)	3,025 (c=40)
QPS @ c=10	947	1,314	1,489
Serial P99 Latency	15.2 ms	9.6 ms	7.8 ms
P99 Latency @ c=1	15.5 ms	9.9 ms	8.1 ms
P99 Latency @ c=100	81.2 ms	49.9 ms	49.6 ms
Recall	96.8%	92.0%	87.9%
nDCG	97.3%	93.1%	89.7%

Metric	#1 M=64 ef_s=192	#2 M=32 ef_s=128	#3 M=24 ef_s=64

1.7.2 k=10

Metric	#1 M=64 ef_s=192	#2 M=32 ef_s=128	#3 M=24 ef_s=64	#4 M=20 ef_s=48
Peak QPS	2,324 (c=100)	4,897 (c=80)	9,719 (c=80)	12,840 (c=100)
QPS @ c=10	1,054	1,602	2,046	2,311
Serial P99 Latency	14.6 ms	8.7 ms	6.0 ms	5.5 ms
P99 Latency @ c=1	14.0 ms	8.5 ms	6.2 ms	5.5 ms
P99 Latency @ c=100	81.0 ms	38.1 ms	18.0 ms	12.3 ms
Recall	99.2%	97.7%	95.0%	92.0%
nDCG	99.1%	97.6%	94.9%	92.0%

1.7.3 Key Observations

- **k=10 throughput scales dramatically with lighter index params:** Exp #4 (M=20, ef_s=48) achieves **12,840 QPS** — **5.5× higher** than Exp #1 (M=64, ef_s=192) — while maintaining 92.0% recall
- **k=100 throughput is more constrained:** Peak QPS varies from 2,345 to 3,025 across configs (~1.3× range), as retrieving more neighbors dominates cost
- **Recall trade-off is modest at k=10:** Recall only drops from 99.2% → 92.0% (−7.2pp) going from heaviest to lightest config, while QPS increases 5.5×
- **Recall trade-off is steeper at k=100:** Recall drops from 96.8% → 87.9% (−8.9pp) for a 1.3× QPS gain
- **Latency improves proportionally:** Serial P99 latency drops from 15.2 ms → 7.8 ms at k=100, and 14.6 ms → 5.5 ms at k=10
- **Saturation point:** Experiments #1–#3 plateau around c=40–80 concurrency; Exp #4 scales further to c=100 before saturating
- **Best balanced config:** Exp #2 (M=32, ef_s=128) offers a good middle ground — 97.7% recall at k=10 with 4,897 QPS, and 92.0% recall at k=100 with 2,975 QPS